NMAM INSTITUTE OF TECHNOLOGY,NITTE

Department of Information Science and Engineering

OPERATING SYSTEMS
PROGRAMMING ASSIGNMENT REPORT

Name:Diya P Shetty

USN:NNM24IS080

Course:Operating System

Github Repository: https://github.com/DiyaPShetty/OS_Task2

SUBMITTED TO

Dr Jason Elroy Martis

Associate Professor

Dept Information Science and Engineering,Nitte

1.Problem Statement and Objectives

1.1 Understanding the Problem

OS make use of synchronization to control processes or threads that share common resources. The problems that caused is as a result of running multiple threads that is race condition and improper access to the shared data [1][2].So Here In this assignment Java has been used to create synchronization problems such as the Dining Philosophers Problem and Readers–Writers Problem with the methods semaphore and monitor [7][8].

It is shown how synchronization enables threads to function in a safe manner and how threads' execution can be tracked using HTML output.

1.2 Goals of the Assignment

The aim of this assignment is to:

- The comprehension about the process and thread synchronization concepts
- Implementation of the Dining Philosophers Problem with the use of semaphores and monitors
- Solution of Readers-Writers Problem by using Semaphores and Monitors
- Prevent multiple use of resources
- The written recording of what happened while the program ran
- HTML files to display the running of the synchronisation programme

2. Readers–Writers: Semaphore-Based Solution

2.1 Background and Purpose

Readers-Writers Problem is a problem which deals with several threads which access the same shared data.Multiple readers may read the shared data simultaneously but the writer should have exclusive rights on reading the data [1].

Semaphore is a concept which helps in controlling the accessing of the shared resources and in coordinating the readers and writers [1] [8].

2.2 Program Design and Development

The solution to the Readers-Writers problem using semaphores is written in this algorithm in Java [8].Synchronization will be employed for controlling access to the shared resource.

This algorithm controls the behavior of the reader-writer and notes some of the important information while the program executes. It becomes easy to know the time at which a particular reader starts or stops reading and writers accessing the shared data.

2.3 Execution Process and Logic

1. A reviewer will asks for authorization to view the shared information
2. Readers may read concurrent data when no writer is changing the data
3. The writer needs to have permission before writing
4. During writing by the writer, access from another competing party is not allowed
5. When the thread has finished the task it releases the lock of the synchronization object

3. Dining Philosophers: Monitor-Based Solution

3.1 Background and Purpose

A monitor is a form of synchronizer that encloses the shared data and the computation occurring on the data.Access to the shared resource is done in a controlled manner [1].

Here, the solution to the Readers-Writers Problem uses Java monitor-based synchronization in place of semaphores [5] [6].

3.2 Program Design and Development

The readers and writers coordination within the program is managed using synchronization of Java [2] [5]. Synchronization provides the way to access the shared variable and also record the events happen in the program.

The advantage of this design pattern is that we can all keep the synchronization code in one place. Moreover, it is also clearer how access control of the shared resource is performed.

3.3 Execution Process and Logic

1. The reader or writer is requesting for access of the shared resource
2. The synchronization algorithm will check whether the operation can go on
3. The readers can read concurrently if the conditions of the implementation allow them to do so
4. The author will have confident access to the shared resource while the changes are being made.
5. Once the operation is completed, the sync algorithm permits any thread that has the go-ahead to continue to do their thing.

   The implementation shows that how to use monitors for thread coordination without handling semaphore operations

4. Dining Philosophers: Semaphore-Based Solution

4.1 Background and Purpose

Dining Philosophers Problem is one of the most popular problems of synchronization in the OS [1]. The philosophers will sit around the table and they will alternatively think and eat. For dining purposes, a philosopher must use two forks, which are to be shared by the other philosophers around him/her.

If In case all philosophers try to take forks simultaneously they will wait forever. This state is called deadlock [1]**.**

4.2 Program Design and Development

This program has solved the Dining Philosophers problem using semaphores in the Java language [8].This synchronization mechanism controls the interaction of the philosophers with the forks so that they would not touch the same fork at the same time.

This program logs the actions undertaken by the philosophers like thinking, waiting for the forks and eating.

4.3 Execution Process and Logic

1. Each philosopher will begin with thinking
2. When a philosopher wants to eat the philosopher will demand the required forks
3. Semaphore controls access to forks
4. Philosophers eat only when they get the required forks
5. When the philosophers are finished, they'll put their forks down.
6. Now some philosophers are going to pick up the released forks and will have other action.

This is how semaphores can be used for synchronization of access to the shared resources. The deadlock occurs on the way to resource acquisition.

5. Dining Philosophers: Monitor-Based Solution

5.1 Background and Purpose

Monitor may also be employed in the solution of dining philosopher's problem [1].Rather than directly employing semaphores to manage each resource a monitor will manage the entire system state and coordination among philosophers.

This would assist in synthesizing the rules for acquiring a fork and for when the Philosopher can eat.

5.2 Program Design and Development

The monitor-based synchronization technique has been applied in order to control the behavior of single philosophers and forks [5] [10]. The log of all events during the program run allows tracking the changes in the philosophers' state.

5.3 Execution Process and Logic

1. The philosopher begins at the thinking level
2. If the thinker seems hungry then the program will check whether or not he is able to eat
3. The philosopher will be able to eat provided that all conditions are met.
4. So if those conditions are not satisfied the philosopher will then wait until that time which he/she is able to eat
5. When the eating is complete, then the philosopher relinquish the forks by changing shared state

As you can see, this implementation demonstrates the use of monitor to manage concurrent access to shared resources.

6. Creating Browser-Based Execution Views

6.1 Background and Purpose

For making the execution of the process clear the output in HTML format will be made from the Java code. Rather than having text displayed in the console the execution events are recorded and an HTML file is created through them.

The humanly perceivable version of the executed activities are included in the HTML file.

6.2 Program Design and Development

The program operates in the following sequence:

1. The Java program will run the code which is responsible for synchronization
2. Events that are deemed to be important are logged in during the process of execution
3. The logs obtained are then transformed into HTML text
4. The program puts the HTML text into a file
5. You can open the created file on any web browser to view the logs

Four different HTML files are created for the four different synchronization implementations.

7. Program Runs and Observations

The four Java applications were compiled and executed separately. The HTML files which were generated were used to observe the events of execution.

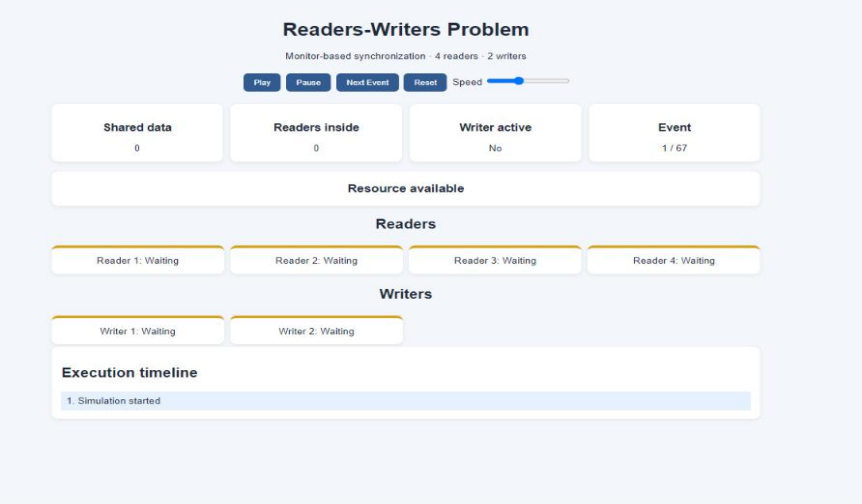
7.1 Readers–Writers Using Semaphore

The program shows how to coordinate readers and writers with semaphores.



7.2 Readers–Writers Using Monitor

Monitor synchronization shows the coordination between readers and writers in the program.



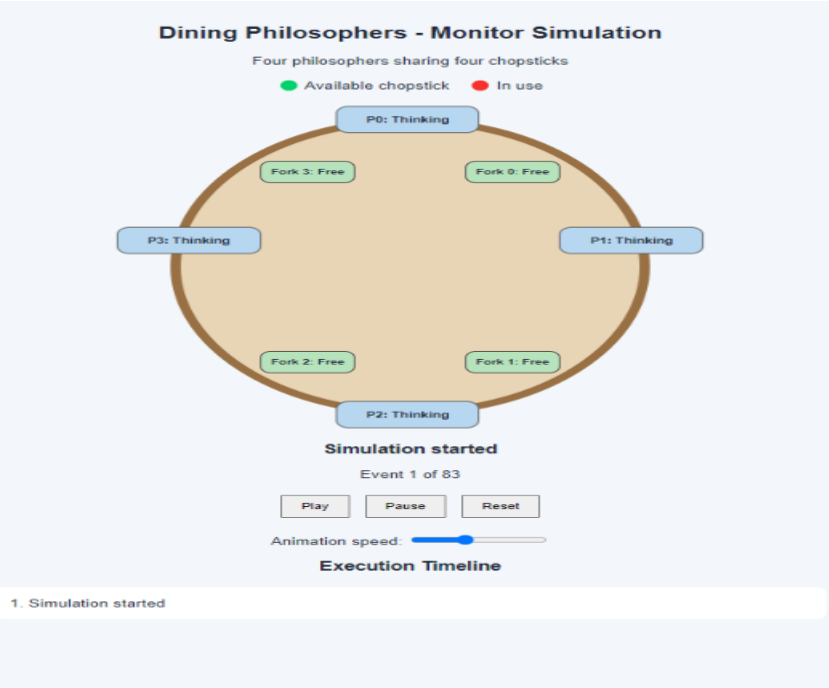
7.3 Dining Philosophers Using Semaphore

It shows how to use semaphores to solve the dining philosophers problem.



7.4 Dining Philosophers Using Monitor

This Program shows the monitor mechanism regulating philosophers states and forks allocation



8. Synchronization Concepts Demonstrated

8.1 Process and Thread Synchronization

It is necessary to synchronize threads that work with common resources [2][4][9]. Synchronization allows for organizing their work and avoiding conflicts.

8.2 Race Condition

The definition of a race condition is that the output of the program depends on the order of thread execution as well as accessing and modifying shared variables [1] [2].

8.3 Semaphore

Semaphore is a tool for synchronization that is utilized to manage the access to shared resources by threads [1], [8]. Threads obtain and release permits as per the rules of synchronization.

8.4 Monitor

Monitors are used to synchronize access to shared resources by threads [1]. Synchronization locks and monitors allow us to control shared resources in Java [3] [5] [6].

8.5 Mutual Exclusion

In mutual exclusion, a lockout means that at some point in time, only one thread has exclusive access to the object or resource [1].

8.6 Deadlock

Deadlock results when programs never end up obtaining all the resources they need to run [1]. The Dining Philosophers Problem illustrates how such a race condition can happen [1].

9. Differences Between the Two Synchronization Methods

Semaphore	Monitor
Uses an explicit acquire or release paradigm [8].	Uses synchronized access to shared data or the methods [3].
Control of semaphore operations is provided by the programmer.	Synchronization logic relies on the shared state and methods.
Can be utilized for gaining access to distributed sources.	Can provide coordination of access to shared resources.
May cause To be in sync. Get sync error as a result of wrong utilization.	May lead to sync errors when used improperly.
Used in the semaphore solutions to both problems.	Used in the monitor solutions to both problems.

Both approaches are suitable for thread synchronization [1]. It is decisions depending on the programming technique used and on resource management requirements.

10. What Was Learned from the Assignment

In this task I have been developed the Dining Philosophers problem and the Readers-Writers problem by employing semaphores and monitors in Java.

While developing these programs I was able to understand the role of synchronization in coordinating threads and managing the shared resources among threads. I also learnt about the mutual exclusion and how incorrect handling of the shared resources may cause the deadlocks.

HTML output generation gave me a better visualization of the execution event log that is generated and gave a clear understanding about the concept.

11. Disclosure of AI Assistance

AI tools has been used as an additional assistant in the completion of this assignment.

ChatGPT (OpenAI): Was used to get an understanding of various concepts in OS synchronization such as semaphores and monitors in the Dining Philosophers Problem and Readers-Writers Problem. This tool also helped me in getting an understanding of Java code as well as resolving programming problems.

Claude (Anthropic): will be used to identify the appropriate references/citations to support the report. Any references/citations provided via this tool will be validated before being used.

AI tools have been employed as aids in the process of learning and preparation of the report. The program implementations and execution results and explanations in the final report are supposed to represent the student's work.

12. Technical Sources Consulted

1. Silberschatz, A., Galvin, P. B., and Gagne, G. *Operating System Concepts*, 10th Edition, Wiley, 2018. https://codex.cs.yale.edu/avi/os-book/OS10/
2. Oracle. *Java Tutorials: Synchronization*. https://docs.oracle.com/javase/tutorial/essential/concurrency/sync.html
3. Oracle. *Java Tutorials: Synchronized Methods*. https://docs.oracle.com/javase/tutorial/essential/concurrency/syncmeth.html
4. Oracle. *Java Tutorials: Thread Objects*. https://docs.oracle.com/javase/tutorial/essential/concurrency/threads.html
5. Oracle. *Java Tutorials: Intrinsic Locks and Synchronization*. https://docs.oracle.com/javase/tutorial/essential/concurrency/locksync.html
6. Oracle. *Java Language Specification, Chapter 17: Threads and Locks*. https://docs.oracle.com/javase/specs/jls/se11/html/jls-17.html
7. Oracle. *Java SE Documentation: Concurrency*. https://docs.oracle.com/en/java/javase/25/core/concurrency.html
8. Oracle. *Java Platform, Standard Edition API Specification: Semaphore*. https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/Semaphore.html
9. Oracle. *Java Platform, Standard Edition API Specification: Thread*. https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Thread.html
10. Oracle. *Java Platform, Standard Edition API Specification: Object*. https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Object.html